

Wade Keith

Large Language Models

Foundations, Systems, Alignment, and
Applications

May 20, 2026

Springer Nature

Preface

This book is written as a modern, research-grounded introduction to large language models. It uses the local course materials as a coverage map, but the prose, organization, notation, examples, and arguments are original. Each chapter is intended to connect three views that are too often taught separately: the mathematical object being optimized, the system that makes the optimization feasible, and the evaluation procedure that determines whether the result is useful.

The book deliberately avoids a repository-by-repository narrative. Instead, it follows the lifecycle of a model: data and tokenization, architecture, pretraining, distributed systems, serving, adaptation, alignment, retrieval, reasoning, multimodality, and safety. Historical systems such as GPT, Alpaca, LoRA, RLHF, and LLaMA are treated as conceptual anchors rather than as recipes to copy without context.

The drafting standard is evidence-first. Claims about algorithms, scaling behavior, evaluation, or deployment are tied to primary papers or technical reports whenever possible. Code from the source repository is used as inspiration for reproducible exercises, not as text to paste into the manuscript.

Declarations, Provenance, and Responsible Use

This manuscript is being prepared from original explanatory text, public research papers, and local course assets provided by the repository owner. Third-party slides, papers, datasets, code repositories, model checkpoints, and media files are treated as source material requiring citation, license review, and permission checks before publication.

The book discusses model training, deployment, alignment, data collection, evaluation, retrieval, and agentic systems. These topics carry risks involving privacy leakage, copyright infringement, unsafe generation, benchmark contamination, dual-use automation, and misleading evaluation. Chapters therefore include explicit provenance and safety notes where those risks are material.

No claim in the manuscript should be interpreted as legal advice, medical advice, security advice, or a recommendation to deploy an unreviewed model in a high-stakes environment.

Contents

Declarations, Provenance, and Responsible Use	vii
Acronyms	xiii
Part I Foundations	
1 What Makes a Language Model Large	3
1.1 The Object of Study	3
1.2 Why Prediction Became a General Interface	5
1.3 Scaling Changed the Research Program	5
1.4 From Base Models to Assistants	6
1.5 Reasoning Makes Inference Part of Training	6
1.6 A Roadmap for the Book	7
1.7 Key Terms	7
1.8 Exercises	8
2 Tokens, Corpora, and Training Signals	9
2.1 Chapter Spine	9
2.2 Core Sections	9
2.2.1 Tokenization as a Modeling Choice	9
2.2.2 Corpus Mixtures and Data Quality	9
2.2.3 Training Signals Beyond Pretraining	10
2.2.4 Provenance and Licensing	10
2.3 Planned Exercises	10
Part II Architectures, Optimization, and Systems	
3 Transformer Mechanics	13
3.1 Chapter Spine	13
3.2 Core Sections	13
3.2.1 Embedding and Positional Information	13
3.2.2 Scaled Dot-Product Attention	13

3.2.3	Masks and Causality	14
3.2.4	Residual Streams and Feed-Forward Blocks	14
3.3	Planned Exercises	14
4	From Transformer to GPT	15
4.1	Chapter Spine	15
4.2	Core Sections	15
4.2.1	Causal Language Modeling	15
4.2.2	Minimal GPT Implementation	15
4.2.3	Decoding	16
4.2.4	Attention Efficiency	16
4.3	Planned Exercises	16
5	LLaMA-Class Architectures	17
5.1	Chapter Spine	17
5.2	Core Sections	17
5.2.1	Normalization and Activations	17
5.2.2	Rotary Position Embeddings	17
5.2.3	KV Cache and Attention Variants	17
5.2.4	Mixture-of-Experts Models	18
5.3	Planned Exercises	18
6	Optimization and Pretraining	19
6.1	Chapter Spine	19
6.2	Core Sections	19
6.2.1	Optimizer Mechanics	19
6.2.2	Stability at Scale	19
6.2.3	Compute-Optimal Planning	20
6.2.4	From Historical Claims to Current Practice	20
6.3	Planned Exercises	20
7	Distributed Training Systems	21
7.1	Chapter Spine	21
7.2	Core Sections	21
7.2.1	Memory Accounting	21
7.2.2	Parallelism Strategies	21
7.2.3	Precision and Communication	22
7.2.4	Operational Reliability	22
7.3	Planned Exercises	22
8	Inference and Serving	23
8.1	Chapter Spine	23
8.2	Core Sections	23
8.2.1	Prefill, Decode, and the KV Cache	23
8.2.2	Batching and Memory Management	23
8.2.3	Compression and Acceleration	24

8.2.4	API Behavior and Load Testing	24
8.3	Planned Exercises	24

Part III Adaptation

9	Supervised Instruction Tuning	27
9.1	Chapter Spine	27
9.2	Core Sections	27
9.2.1	Instruction Data	27
9.2.2	Synthetic Data	27
9.2.3	Training and Evaluation	28
9.2.4	Limits of SFT	28
9.3	Planned Exercises	28
10	Parameter-Efficient Adaptation	29
10.1	Chapter Spine	29
10.2	Core Sections	29
10.2.1	Adapters and Prefix Methods	29
10.2.2	LoRA	29
10.2.3	QLoRA and Quantized Training	29
10.2.4	Evaluation of Adaptation	30
10.3	Planned Exercises	30
11	Domain and Language Adaptation	31
11.1	Chapter Spine	31
11.2	Core Sections	31
11.2.1	Language Adaptation	31
11.2.2	Domain Knowledge	31
11.2.3	Structured Tasks	32
11.2.4	Safety in Domains	32
11.3	Planned Exercises	32

Part IV Alignment, Applications, and Evaluation

12	Retrieval, Tools, and Agents	35
12.1	Chapter Spine	35
12.2	Core Sections	35
12.2.1	Retrieval Pipelines	35
12.2.2	Generation with Evidence	35
12.2.3	Tool Use	36
12.2.4	Agentic Workflows	36
12.3	Planned Exercises	36

13	Preference Learning and Alignment	37
13.1	Chapter Spine	37
13.2	Core Sections	37
13.2.1	Reward Models	37
13.2.2	RLHF with PPO	37
13.2.3	Direct Preference Optimization	38
13.2.4	Safety and Refusal	38
13.3	Planned Exercises	38
14	Reasoning and Test-Time Compute	39
14.1	Chapter Spine	39
14.2	Core Sections	39
14.2.1	Prompted Reasoning	39
14.2.2	Search and Verification	39
14.2.3	Reinforcement Learning with Verifiable Rewards	40
14.2.4	Inference-Time Scaling	40
14.3	Planned Exercises	40
15	Multimodal LLMs	41
15.1	Chapter Spine	41
15.2	Core Sections	41
15.2.1	Contrastive Pretraining	41
15.2.2	Connecting Vision and Language	41
15.2.3	Instruction Tuning for VLMs	42
15.2.4	Evaluation and Safety	42
15.3	Planned Exercises	42
16	Evaluation, Safety, and Governance	43
16.1	Chapter Spine	43
16.2	Core Sections	43
16.2.1	Benchmarks and Their Failure Modes	43
16.2.2	Human Evaluation	43
16.2.3	Hallucination and Robustness	44
16.2.4	Governance	44
16.3	Planned Exercises	44
A	Appendix: Reproducibility and Notation Conventions	45
A.1	Notation	45
A.2	Experiment Records	45
A.3	Source Provenance	45
	Glossary	47
	References	48

Acronyms

API	Application programming interface.
BPE	Byte-pair encoding.
CoT	Chain-of-thought prompting.
DPO	Direct preference optimization.
FFN	Feed-forward network.
FSDP	Fully sharded data parallelism.
GQA	Grouped-query attention.
KV	Key-value.
LLM	Large language model.
LoRA	Low-rank adaptation.
MoE	Mixture of experts.
MQA	Multi-query attention.
PEFT	Parameter-efficient fine-tuning.
PPO	Proximal policy optimization.
RAG	Retrieval-augmented generation.
RLHF	Reinforcement learning from human feedback.
RLVR	Reinforcement learning with verifiable rewards.
RoPE	Rotary position embedding.
SFT	Supervised fine-tuning.
VLM	Vision-language model.

Part I

Foundations

Chapter 1

What Makes a Language Model Large

Abstract Large language models are not defined by parameter count alone. They are coupled artifacts whose behavior emerges from data selection, tokenization, architecture, optimization, post-training, inference systems, and evaluation. This chapter establishes that lifecycle view. It explains why causal prediction became the dominant training interface, why scaling laws changed the engineering economics of language modeling, why instruction following and preference learning turned pretrained models into assistant systems, and why reasoning models made inference-time computation part of the modeling problem.

1.1 The Object of Study

A language model assigns probabilities to token sequences, but a modern large language model is better understood as a trained conditional computation system whose distribution is shaped by corpus design, optimizer dynamics, architectural constraints, and deployment-time decoding. For a tokenized sequence $x_{1:T}$, the causal language-model factorization is

$$p_{\theta}(x_{1:T}) = \prod_{t=1}^T p_{\theta}(x_t \mid x_{<t}), \quad (1.1)$$

and the usual training objective minimizes the average negative log-likelihood,

$$\mathcal{L}(\theta) = -\frac{1}{T} \sum_{t=1}^T \log p_{\theta}(x_t \mid x_{<t}), \quad (1.2)$$

with perplexity defined as $\exp(\mathcal{L})$ when the loss is measured in natural-log units. The Transformer made this system practical at scale because self-attention exposes long-range token interactions while remaining highly parallel on accelerators [29]. GPT-style causal models then showed that a single left-to-right objective can support broad transfer when the model is trained on enough heterogeneous text [22, 3]. BERT-style masked

Table 1.1 Axes of largeness in modern language-model systems.

Axis	Symbol	What Scales	Typical Failure Mode
Parameters	N	Dense weights or total MoE weights	Capacity improves while memory, bandwidth, and overfitting risks grow
Active compute	N_{active}	Parameters used per token in sparse models	Routing imbalance or expert under-use hides nominal scale
Training data	D	Tokens, documents, domains, and languages	More tokens amplify duplication, leakage, and licensing problems
Training compute	C	FLOPs and accelerator time	Undertraining or wasted compute from poor data/model balance
Context	T	Prompt and generated sequence length	KV-cache memory and attention cost dominate serving
Inference budget	B	Samples, search, retrieval, tools, verifiers	Higher pass rates can hide weak calibration or invalid verifiers

Data and provenance → tokenizer → architecture → objective and optimizer → distributed training → post-training → inference system → evaluation and governance

Fig. 1.1 The lifecycle stack used throughout the book. Each arrow represents a contract: the next layer inherits the assumptions, artifacts, and failure modes of the previous layer.

models remain important for representation learning and retrieval, and dense retrieval systems later made representation quality central to open-domain question answering and RAG pipelines [9, 16, 19].

The word “large” therefore refers to several coupled axes, summarized in Table 1.1. Parameter count changes capacity, but active parameters matter separately in sparse mixture-of-experts systems; token count changes the statistical evidence seen during training, but token provenance determines whether the evidence is useful or publishable; context length changes what can be conditioned on, but KV-cache memory and attention cost determine whether that context can be served economically [15, 13, 7]. Inference budget is now also part of model behavior, because retrieval, reranking, sampling, verification, tool calls, and explicit reasoning traces can spend additional computation after pretraining has ended [2, 19, 4, 33].

This book treats an large language model as a stack rather than as a single neural network. The stack begins with raw data and a tokenizer; passes through architecture, loss, optimizer, and distributed training; continues through supervised instruction tuning and preference optimization; and ends in an inference service that must decide how much context, retrieval, search, and safety filtering to apply. A mistake in any layer can be misread as a model limitation: a weak tokenizer can harm multilingual coverage, a contaminated benchmark can exaggerate capability, an unstable optimizer can waste compute, and a poorly designed serving path can make a strong model unusable under latency constraints [25, 32].

1.2 Why Prediction Became a General Interface

The central technical bargain of language modeling is that next-token prediction converts unstructured text into a dense self-supervised training signal. Every token in a document can contribute a target, so web-scale corpora become usable without manual labels [3]. The bargain is imperfect: the objective rewards distributional continuation, not truth, helpfulness, calibration, or harmlessness [21]. Yet it is powerful because language co-occurs with facts, code, dialogue, procedures, arguments, and instructions, so a model trained to predict text can internalize many latent tasks before any task-specific fine-tuning [3].

The Transformer architecture amplified this bargain. Self-attention gives each position a data-dependent path to previous positions, so the model can condition on both local syntax and distant context [29]. Multi-head attention gives the model multiple learned projection subspaces for comparing and aggregating tokens, while feed-forward sublayers supply high-capacity tokenwise transformations. Residual pathways and normalization make very deep stacks trainable, and modern decoder families refine these ingredients with rotary position embeddings, RMS normalization, gated feed-forward networks, grouped-query attention, and mixture-of-experts routing [26, 27, 1, 7].

Prediction also gives a clean implementation path. The model receives token ids, maps them to embeddings, repeatedly mixes information through attention and feed-forward blocks, and projects the final hidden states to vocabulary logits. Training minimizes cross-entropy between predicted logits and the next token, while generation samples or selects tokens from the resulting distribution. Later chapters derive this computation from tensor shapes, then connect each mathematical object to original exercises in the companion implementation repository after licensing and provenance checks.

1.3 Scaling Changed the Research Program

Before the modern scaling era, natural language processing was organized around task-specific datasets, architectures, and losses. Scaling-law work reframed the field by showing that loss follows empirical power laws over model size, dataset size, and compute across broad ranges [15]. Compute-optimal training then shifted attention from simply increasing parameters to balancing model size and token count, arguing that many early large models were undertrained relative to their compute budgets [13]. The consequence is practical: a serious training plan must budget parameters, tokens, sequence length, hardware throughput, and data quality together.

Open model reports made this systems view explicit, although they should be read as technical reports rather than as a substitute for independent replication. LLaMA demonstrated that carefully trained smaller models can be competitive when token budgets and data mixtures are chosen well [27]. Llama 2 expanded the public discussion of pretraining, supervised fine-tuning, preference data, and safety evaluation [28]. Llama 3 pushed the recipe toward longer context, multilingual coverage, coding, safety models,

and extensive post-training [11]. DeepSeek-V3 and Kimi K2 indicate that frontier-scale open-weight systems increasingly combine mixture-of-experts routing, specialized attention mechanisms, and agentic or coding-oriented post-training, but their claims still require careful benchmark and deployment-context interpretation [7, 17].

Scaling also exposed failure modes. Training loss can improve while benchmark validity degrades through leakage, contamination, or overfitting to public leaderboards [25, 32]. Larger context windows can increase cost faster than they increase robust long-context reasoning. More samples at inference time can improve pass rates while hiding weak calibration or brittle verification. This book therefore treats evaluation as a modeling component, not as an afterthought.

1.4 From Base Models to Assistants

A pretrained base model is not yet an assistant. It has learned broad continuation behavior, but user-facing behavior requires additional training signals that specify what counts as a helpful response. Supervised instruction tuning supplies demonstrations of desired behavior, and RLHF adds preference rankings so the model can be optimized toward responses humans prefer under a specific annotation protocol [21]. Direct preference optimization and related methods simplify this stage by fitting preferences without an explicit online reinforcement-learning loop, although they still depend on the quality and scope of the preference data [23].

Parameter-efficient adaptation changed who can perform this second stage. LoRA freezes the base model and learns low-rank updates inside selected linear layers, which makes adaptation far cheaper than full fine-tuning for many settings [14]. QLoRA combines low-rank adaptation with quantized base weights, making instruction tuning feasible on much smaller hardware budgets [8]. These methods are not magic compression; they change the trainable subspace, memory footprint, optimizer state, and failure modes. Chapter 10 treats rank, target modules, quantization error, and evaluation as design decisions rather than as fixed defaults.

Assistants also need retrieval and tools when the answer depends on fresh, private, or long-tail knowledge. Retrieval-augmented generation inserts external evidence into the context, while tool-use patterns let the model call search, calculators, databases, or code execution environments [19, 35]. These systems trade parametric memory for runtime grounding, but they introduce their own failure modes: retriever recall, reranker precision, context ordering, citation faithfulness, and prompt-injection risk.

1.5 Reasoning Makes Inference Part of Training

Reasoning in large language models is not a single mechanism. Chain-of-thought prompting elicits intermediate natural-language steps, self-consistency samples multiple paths, tree search branches over candidate partial solutions, and verifier-guided methods score or filter reasoning traces [30, 34]. These traces can improve task perfor-

mance without being faithful causal explanations of the model’s internal computation, so they must be evaluated as generated artifacts rather than accepted as transparent reasoning records [20]. Reasoning-oriented reinforcement learning adds another layer by rewarding correct final answers, verified intermediate steps, or task-specific executable outcomes [6, 31]. The strongest post-2024 lesson is that inference-time compute and training-time incentives must be studied together, because a model can appear more capable simply by searching longer over a learned distribution [4, 33].

This perspective changes how we teach algorithms such as PPO, DPO, GRPO-style training, process reward models, and Monte Carlo tree search. The book does not present reinforcement learning as a detached prerequisite followed by an unrelated RLHF recipe. Instead, it follows the sequence-level control problem: what is being rewarded, what distribution is being constrained, what verifier is trusted, and how much extra computation is spent at inference time.

1.6 A Roadmap for the Book

Part I defines the foundation: what is being modeled, how text becomes tokens, how corpora become training signals, and why provenance matters. Part II builds the architecture and systems spine, moving from attention and GPT-style decoders to LLaMA-class blocks, optimization, distributed training, and inference serving. Part III studies adaptation through instruction tuning, synthetic data, LoRA, QLoRA, domain adaptation, and multilingual specialization. Part IV covers retrieval, tools, agents, preference learning, reasoning, multimodality, evaluation, safety, and governance.

The intended reading discipline is simple. Each chapter starts from a concrete problem, states the mathematical object being optimized, connects the object to implementation, and then asks how the claim would be evaluated. That discipline is the main defense against treating course slides, benchmark tables, or model cards as self-evident truth. The goal is not to memorize the names of model families; it is to understand why a modeling choice works, when it fails, and how to test it in a reproducible system.

1.7 Key Terms

Base model	A pretrained model before instruction tuning, preference optimization, or task-specific adaptation.
Context length	The number of tokens available to condition the model during a forward pass.
Inference budget	The computation spent after a prompt is received, including decoding, retrieval, search, verification, and tool use.
Perplexity	The exponential of average negative log-likelihood under a language model, useful for comparing models only when tokenization and evaluation data are controlled.

Post-training	Training after base pretraining, including SFT, preference optimization, safety tuning, and domain adaptation.
---------------	--

1.8 Exercises

1. Write the causal factorization and cross-entropy loss for a batch with shape $B \times T$. State which tokens should be masked for a chat-style SFT example.
2. Pick two rows from Table 1.1. For each row, describe one benchmark result that could improve while the real system becomes worse.
3. Explain why a model with fewer total parameters can be more expensive to serve than a larger model under a different context length or KV-cache layout.
4. Design a provenance checklist for one dataset, one code repository, one figure, and one model checkpoint before they can be used in this book.
5. Give an example where a chain-of-thought answer is correct but the explanation is not a faithful record of the underlying computation.

Chapter 2

Tokens, Corpora, and Training Signals

Abstract This chapter turns raw text into a defensible training signal. It covers tokenization, corpus construction, deduplication, filtering, contamination control, packing, licensing, and the difference between pretraining, instruction, preference, retrieval, and evaluation data.

2.1 Chapter Spine

Data is not a passive input to an large language model; it is the main specification of what the model can imitate, remember, refuse, and hallucinate. This chapter will connect byte-level and subword tokenization to multilingual coverage, sequence packing to compute efficiency, and corpus provenance to benchmark validity.

2.2 Core Sections

2.2.1 Tokenization as a Modeling Choice

BPE, unigram, byte fallback, vocabulary extension, multilingual token efficiency, and tokenizer-model coupling.

2.2.2 Corpus Mixtures and Data Quality

Web text, books, code, math, dialogue, domain corpora, decontamination, deduplication, and mixture weighting.

2.2.3 Training Signals Beyond Pretraining

Instruction data, preference data, verifier data, tool traces, retrieval corpora, and evaluation sets.

2.2.4 Provenance and Licensing

Copyright, privacy, dataset cards, model cards, and publication-safe reuse of local course assets.

2.3 Planned Exercises

Build a small tokenizer, measure token efficiency across languages, and run a contamination check between a toy training corpus and a held-out benchmark.

Part II
Architectures, Optimization, and Systems

Chapter 3

Transformer Mechanics

Abstract This chapter derives the Transformer from tensor shapes. It explains attention, masking, residual streams, normalization, feed-forward layers, positional information, and the implementation invariants needed to debug a model from first principles.

3.1 Chapter Spine

The Transformer is a sequence-to-sequence computation graph whose core operation is content-dependent mixing. The reader should leave this chapter able to predict every major tensor shape in a forward pass and to explain why masking, normalization, and residual connections are necessary.

3.2 Core Sections

3.2.1 Embedding and Positional Information

Token embeddings, learned and sinusoidal positions, rotary embeddings, and where position enters attention.

3.2.2 Scaled Dot-Product Attention

Queries, keys, values, attention scores, softmax, dropout, and numerical stability.

3.2.3 Masks and Causality

Padding masks, causal masks, attention bias, and encoder-decoder cross-attention.

3.2.4 Residual Streams and Feed-Forward Blocks

Layer normalization, pre-norm versus post-norm, MLP expansion, gated activations, and residual information flow.

3.3 Planned Exercises

Implement single-head attention, multi-head attention, and a masked decoder block with shape assertions at every step.

Chapter 4

From Transformer to GPT

Abstract This chapter specializes the Transformer into a causal decoder. It covers next-token loss, GPT-style blocks, sampling, repetition control, loss curves, minimal training loops, and efficient attention as used in practical GPT implementations.

4.1 Chapter Spine

GPT is not merely a Transformer with a different name; it is a causal training and generation interface. The chapter moves from the decoder block to cross-entropy training, then from logits to sampling behavior.

4.2 Core Sections

4.2.1 Causal Language Modeling

Teacher forcing, shifted labels, cross-entropy, perplexity, and batch packing.

4.2.2 Minimal GPT Implementation

Configuration, block stack, tied embeddings, optimizer groups, and checkpointing.

4.2.3 Decoding

Greedy decoding, temperature, top- k , nucleus sampling, repetition penalties, and stop conditions.

4.2.4 Attention Efficiency

FlashAttention, memory bandwidth, IO-awareness, and why exact attention can be made faster without changing model semantics [5].

4.3 Planned Exercises

Train a toy character-level model, reproduce a loss curve, and compare sampling controls on the same prompt.

Chapter 5

LLaMA-Class Architectures

Abstract This chapter studies the architectural conventions used by modern open decoder models: RMSNorm, RoPE, SwiGLU-style feed-forward networks, grouped-query attention, KV cache design, long-context extensions, and mixture-of-experts variants.

5.1 Chapter Spine

Modern open LLMs are mostly decoder-only Transformers, but their engineering details matter. LLaMA-class models changed the default block, attention cache, tokenizer, and post-training expectations [27, 28, 11].

5.2 Core Sections

5.2.1 Normalization and Activations

RMSNorm, pre-normalization, SwiGLU, and activation-memory accounting.

5.2.2 Rotary Position Embeddings

RoPE geometry, extrapolation pitfalls, and long-context scaling.

5.2.3 KV Cache and Attention Variants

Multi-head, multi-query, grouped-query, and latent attention variants [24, 1, 7].

5.2.4 Mixture-of-Experts Models

Routing, expert capacity, load balancing, activated parameters, and failure modes in sparse computation.

5.3 Planned Exercises

Modify a GPT block into a LLaMA-style block and compare parameter count, KV cache size, and forward-pass memory.

Chapter 6

Optimization and Pretraining

Abstract This chapter links scaling laws to pretraining recipes. It covers objective design, AdamW, learning-rate schedules, warmup, gradient clipping, stability, checkpointing, data order, loss prediction, and compute-optimal planning.

6.1 Chapter Spine

Pretraining is a joint optimization over model size, token budget, data mixture, precision, and hardware throughput. Scaling laws provide planning tools, but they do not remove the need for careful loss monitoring and failure analysis [15, 13].

6.2 Core Sections

6.2.1 Optimizer Mechanics

AdamW, weight decay, gradient clipping, warmup, cosine decay, and parameter groups.

6.2.2 Stability at Scale

Initialization, normalization, loss spikes, mixed precision, checkpoint recovery, and monitoring.

6.2.3 Compute-Optimal Planning

Parameter-token tradeoffs, FLOPs estimates, sequence length, and undertraining.

6.2.4 From Historical Claims to Current Practice

What older scaling laws explain well, what later model reports changed, and where empirical judgment still dominates.

6.3 Planned Exercises

Estimate the compute budget for several model-token configurations and compare the resulting memory and throughput constraints.

Chapter 7

Distributed Training Systems

Abstract This chapter covers the systems layer that makes large-scale training possible: data parallelism, tensor parallelism, pipeline parallelism, fully sharded training, ZeRO-style optimizer partitioning, activation checkpointing, NCCL communication, and throughput accounting.

7.1 Chapter Spine

Training systems determine whether a model recipe is feasible. The chapter uses memory and bandwidth accounting as the bridge between algorithmic intent and hardware execution.

7.2 Core Sections

7.2.1 Memory Accounting

Parameters, gradients, optimizer states, activations, temporary buffers, and fragmentation.

7.2.2 Parallelism Strategies

DDP, FSDP, ZeRO, tensor parallelism, pipeline parallelism, expert parallelism, and hybrid layouts.

7.2.3 Precision and Communication

FP32, BF16, FP16, FP8, loss scaling, collectives, overlap, and network topology.

7.2.4 Operational Reliability

Checkpoint cadence, resumption, straggler detection, data-loader bottlenecks, and experiment tracking.

7.3 Planned Exercises

Compute per-GPU memory for a small decoder and explain which parallelism strategy fixes each bottleneck.

Chapter 8

Inference and Serving

Abstract This chapter studies LLM inference as a systems problem. It covers prefill versus decode, KV caches, batching, paged attention, quantization, speculative decoding, streaming APIs, load testing, and latency-throughput tradeoffs.

8.1 Chapter Spine

Serving is not just running the training model in evaluation mode. Autoregressive decoding changes the compute shape, and user-facing systems must manage memory, queues, context length, and streaming behavior under load.

8.2 Core Sections

8.2.1 Prefill, Decode, and the KV Cache

Why prompt processing and token generation stress different resources.

8.2.2 Batching and Memory Management

Continuous batching, paged attention, cache eviction, and fragmentation [18].

8.2.3 Compression and Acceleration

Weight-only quantization, activation quantization, speculative decoding, and hardware-specific kernels.

8.2.4 API Behavior and Load Testing

Streaming responses, request cancellation, timeout policy, tokens per second, requests per second, and tail latency.

8.3 Planned Exercises

Design a load-test table that reports throughput, average latency, tail latency, and cost per million tokens.

Part III

Adaptation

Chapter 9

Supervised Instruction Tuning

Abstract This chapter explains how pretrained models become instruction-following models through supervised examples. It covers chat templates, task diversity, Alpaca-style self-instruction, synthetic data, refusal data, data quality, and the limits of imitation.

9.1 Chapter Spine

Instruction tuning changes the interface a model expects. The chapter separates the historical importance of Alpaca-style recipes from the modern requirements of dataset auditing, chat formatting, safety coverage, and evaluation.

9.2 Core Sections

9.2.1 Instruction Data

Prompt-response pairs, multi-turn dialogue, system messages, tool traces, and formatting contracts.

9.2.2 Synthetic Data

Self-Instruct, distillation from stronger models, diversity filters, and error amplification.

9.2.3 Training and Evaluation

Loss masking, sequence packing, held-out tasks, refusal behavior, and benchmark leakage.

9.2.4 Limits of SFT

Why imitation alone does not produce calibrated, harmless, or preference-aligned behavior [21].

9.3 Planned Exercises

Convert a raw instruction dataset into a chat template and inspect which tokens should contribute to loss.

Chapter 10

Parameter-Efficient Adaptation

Abstract This chapter covers adapters, prefix tuning, prompt tuning, LoRA, QLoRA, target-module selection, rank choice, quantization, domain adaptation tradeoffs, and evaluation of adapted models.

10.1 Chapter Spine

Parameter-efficient fine-tuning reduces the cost of adaptation by restricting what changes. The important question is not only whether a method trains, but what subspace it can express and what deployment cost it adds.

10.2 Core Sections

10.2.1 Adapters and Prefix Methods

Where trainable parameters enter the network and how they affect inference.

10.2.2 LoRA

Low-rank updates, scaling, rank, target modules, merge behavior, and interference [14].

10.2.3 QLoRA and Quantized Training

Quantized base weights, optimizer states, memory savings, and quantization error [8].

10.2.4 Evaluation of Adaptation

Task metrics, regression tests, forgetting, multilingual effects, and safety drift.

10.3 Planned Exercises

Fine-tune a small model with LoRA, vary rank and target modules, and report both task accuracy and trainable parameter count.

Chapter 11

Domain and Language Adaptation

Abstract This chapter studies how LLMs are adapted to Chinese, domain-specific corpora, NL2SQL, medical-style retrieval tasks, and other specialized settings. It compares continual pretraining, supervised fine-tuning, retrieval, tokenizer extension, and evaluation under domain shift.

11.1 Chapter Spine

Domain adaptation is not one technique. The right choice depends on whether the missing capability is vocabulary coverage, factual knowledge, style, reasoning format, schema grounding, or access to private documents.

11.2 Core Sections

11.2.1 Language Adaptation

Tokenizer extension, bilingual corpora, Chinese LLaMA-style workflows, and multilingual evaluation.

11.2.2 Domain Knowledge

Continual pretraining versus retrieval versus SFT for specialized knowledge.

11.2.3 Structured Tasks

NL2SQL, schema linking, execution accuracy, and dataset leakage.

11.2.4 Safety in Domains

Medical, legal, financial, and organizational settings as high-stakes contexts requiring stronger evaluation.

11.3 Planned Exercises

Compare a retrieval-only baseline with a LoRA-adapted model on a small domain QA set.

Part IV
Alignment, Applications, and Evaluation

Chapter 12

Retrieval, Tools, and Agents

Abstract This chapter covers retrieval-augmented generation, embeddings, vector databases, retrievers, rerankers, chunking, citation faithfulness, tool calls, ReAct-style reasoning-action loops, and agent workflow evaluation.

12.1 Chapter Spine

RAG is a control system around an large language model, not a prompt trick. It requires retriever recall, reranker precision, context budgeting, answer synthesis, citation discipline, and defense against prompt injection.

12.2 Core Sections

12.2.1 Retrieval Pipelines

Chunking, embeddings, vector search, sparse retrieval, hybrid retrieval, reranking, and freshness.

12.2.2 Generation with Evidence

Context ordering, attribution, answer abstention, and grounded response criteria [19, 10].

12.2.3 Tool Use

Function calls, calculators, databases, code execution, and error recovery.

12.2.4 Agentic Workflows

ReAct, planning, memory, multi-step tool use, and evaluation of task completion [35, 17].

12.3 Planned Exercises

Build a small RAG system, evaluate retrieval recall separately from answer faithfulness, and test a prompt-injection case.

Chapter 13

Preference Learning and Alignment

Abstract This chapter explains reward modeling, RLHF, PPO, KL regularization, direct preference optimization, reference-free variants, annotation protocols, safety data, and the gap between preference learning and true alignment.

13.1 Chapter Spine

Alignment methods optimize models toward a stated preference signal. The hard parts are defining the signal, controlling distribution shift, avoiding reward hacking, and measuring whether the resulting behavior is actually better for the intended users.

13.2 Core Sections

13.2.1 Reward Models

Pairwise rankings, Bradley-Terry losses, calibration, annotator disagreement, and overoptimization.

13.2.2 RLHF with PPO

Policy, reference model, KL penalty, reward shaping, value estimates, and rollout cost [21].

13.2.3 Direct Preference Optimization

DPO, reference models, implicit rewards, and stability tradeoffs [23].

13.2.4 Safety and Refusal

Helpfulness-harmlessness tensions, jailbreak resistance, red-teaming, and refusal over-generalization.

13.3 Planned Exercises

Train a toy reward model from pairwise preferences and inspect how changing the KL coefficient changes policy behavior.

Chapter 14

Reasoning and Test-Time Compute

Abstract This chapter covers chain-of-thought prompting, self-consistency, process supervision, verifiers, tree search, MCTS-style reasoning, RLVR, GRPO-style training, DeepSeek-R1-style pipelines, and the limits of spending more inference compute.

14.1 Chapter Spine

Reasoning methods allocate computation across steps, samples, branches, or verifiers. The chapter treats them as algorithms with budgets and failure modes, not as proof that a model has human-like reasoning.

14.2 Core Sections

14.2.1 Prompted Reasoning

Chain-of-thought, self-consistency, decomposition, and when explicit traces help [30].

14.2.2 Search and Verification

Tree-of-thought, MCTS, process reward models, and verifier validity [34].

14.2.3 Reinforcement Learning with Verifiable Rewards

RLVR, GRPO-style objectives, reward hacking, and reasoning-boundary evaluation [6, 31].

14.2.4 Inference-Time Scaling

Sequential, parallel, and tree-structured compute; budget forcing; and limitations on knowledge-intensive tasks [4, 33].

14.3 Planned Exercises

Compare single-sample answers, self-consistency, and a simple verifier loop on a small math or code benchmark.

Chapter 15

Multimodal LLMs

Abstract This chapter introduces multimodal LLMs through CLIP-style representation learning, visual encoders, projection layers, cross-attention, multimodal instruction tuning, image-text evaluation, and safety issues unique to multimodal systems.

15.1 Chapter Spine

Multimodal models add another alignment problem: continuous signals such as images, audio, video, or sensor data must be represented in a form a language model can use without losing task-relevant structure.

15.2 Core Sections

15.2.1 Contrastive Pretraining

CLIP-style image-text alignment and representation transfer.

15.2.2 Connecting Vision and Language

Frozen encoders, projection modules, cross-attention, token compression, and end-to-end training.

15.2.3 Instruction Tuning for VLMs

Image-grounded dialogue, OCR, chart reasoning, visual hallucination, and annotation quality.

15.2.4 Evaluation and Safety

Multimodal benchmarks, adversarial images, privacy, and domain-specific constraints [12].

15.3 Planned Exercises

Trace an image through a frozen vision encoder and projection layer into a language-model prompt.

Chapter 16

Evaluation, Safety, and Governance

Abstract This chapter closes the book by treating evaluation, safety, deployment governance, provenance, privacy, copyright, benchmark contamination, hallucination, interpretability, model cards, and operational monitoring as part of the LLM engineering discipline.

16.1 Chapter Spine

Evaluation is the only way to know whether a modeling decision helped. Safety and governance are the constraints that determine whether the resulting system should be deployed.

16.2 Core Sections

16.2.1 Benchmarks and Their Failure Modes

Accuracy, pass@k, judge models, contamination, benchmark saturation, and task mismatch.

16.2.2 Human Evaluation

Rubrics, pairwise preference, annotator disagreement, calibration, and cost.

16.2.3 Hallucination and Robustness

Factuality, abstention, uncertainty, retrieval faithfulness, adversarial prompts, and distribution shift.

16.2.4 Governance

Model cards, dataset cards, logging, privacy, copyright, incident response, and high-stakes deployment limits.

16.3 Planned Exercises

Write an evaluation card for a small LLM application, including success metrics, disallowed uses, and known blind spots.

Appendix A

Appendix: Reproducibility and Notation Conventions

A.1 Notation

Vectors are written in bold lowercase, matrices in uppercase, and tensors with named shapes when ambiguity would otherwise obscure the implementation. Sequence length is denoted by T , hidden width by d_{model} , attention heads by h , and vocabulary size by V .

A.2 Experiment Records

Every executable experiment in the final manuscript should state the model, dataset, tokenizer, hardware, random seed policy, expected runtime, primary metric, and known failure modes.

A.3 Source Provenance

Local course assets are used as a map of topics and implementation exercises. Research claims should be cited to primary papers, and code-derived exercises should include license checks before publication.

Glossary

Attention	A content-dependent mixing operation that forms query, key, and value representations and aggregates values according to query-key compatibility.
Context window	The maximum number of tokens a model can condition on during a forward pass.
Instruction tuning	Fine-tuning a pretrained model on examples that pair user-like instructions with target responses.
KV cache	Stored key and value tensors from previous decoding steps, reused to avoid recomputing attention over the prefix.
Preference model	A model or implicit objective that represents which of two candidate responses should be preferred under a target policy or annotation protocol.
Retrieval-augmented generation	A generation pattern in which external documents are retrieved and inserted into the model context before or during answer generation.
Test-time compute	Additional computation spent at inference time through longer reasoning traces, multiple samples, verification, search, or refinement.

References

1. Ainslie, J., Lee-Thorp, J., de Jong, M., Zemlyanskiy, Y., Lebron, F., Sanghai, S.: Gqa: Training generalized multi-query transformer models from multi-head checkpoints. arXiv preprint arXiv:2305.13245 (2023). URL <https://arxiv.org/abs/2305.13245>
2. Borgeaud, S., Mensch, A., Hoffmann, J., Cai, T., Rutherford, E., Millican, K., van den Driessche, G., Lespiau, J.B., Damoc, B., Clark, A., et al.: Improving language models by retrieving from trillions of tokens. arXiv preprint arXiv:2112.04426 (2022). URL <https://arxiv.org/abs/2112.04426>
3. Brown, T.B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al.: Language models are few-shot learners. arXiv preprint arXiv:2005.14165 (2020). URL <https://arxiv.org/abs/2005.14165>
4. Chen, Z., Wang, S., Tan, Z., Fu, X., Lei, Z., Wang, P., Liu, H., Shen, C., Li, J.: A survey of scaling in large language model reasoning. arXiv preprint arXiv:2504.02181 (2025). URL <https://arxiv.org/abs/2504.02181>
5. Dao, T., Fu, D.Y., Ermon, S., Rudra, A., Re, C.: Flashattention: Fast and memory-efficient exact attention with io-awareness. arXiv preprint arXiv:2205.14135 (2022). URL <https://arxiv.org/abs/2205.14135>
6. DeepSeek-AI, Guo, D., Yang, D., Zhang, H., Song, J., Wang, P., Xu, R., Zhu, Q., Zhang, R., Ma, S., et al.: Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. arXiv preprint arXiv:2501.12948 (2025). URL <https://arxiv.org/abs/2501.12948>
7. DeepSeek-AI, Liu, A., Feng, B., Xue, B., Wang, B., Wu, B., Lu, C., Zhao, C., Deng, C., Ruan, C., et al.: Deepseek-v3 technical report. arXiv preprint arXiv:2412.19437 (2024). URL <https://arxiv.org/abs/2412.19437>
8. Dettmers, T., Pagnoni, A., Holtzman, A., Zettlemoyer, L.: Qlora: Efficient finetuning of quantized llms. arXiv preprint arXiv:2305.14314 (2023). URL <https://arxiv.org/abs/2305.14314>
9. Devlin, J., Chang, M.W., Lee, K., Toutanova, K.: Bert: Pre-training of deep bidirectional transformers for language understanding. arXiv preprint arXiv:1810.04805 (2018). URL <https://arxiv.org/abs/1810.04805>
10. Gan, A., Yu, H., Zhang, K., Liu, Q., Yan, W., Huang, Z., Tong, S., Hu, G.: Retrieval augmented generation evaluation in the era of large language models: A comprehensive survey. arXiv preprint arXiv:2504.14891 (2025). URL <https://arxiv.org/abs/2504.14891>
11. Grattafiori, A., Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Vaughan, A., et al.: The llama 3 herd of models. arXiv preprint arXiv:2407.21783 (2024). URL <https://arxiv.org/abs/2407.21783>
12. Han, L., Mubarak, A., Baimagambetov, A., Polatidis, N., Baker, T.: Multimodal large language models: A survey. arXiv preprint arXiv:2506.10016 (2025). URL <https://arxiv.org/abs/2506.10016>
13. Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., Cai, T., Rutherford, E., de Las Casas, D., Hendricks, L.A., Welbl, J., Clark, A., et al.: Training compute-optimal large language models. arXiv preprint arXiv:2203.15556 (2022). URL <https://arxiv.org/abs/2203.15556>
14. Hu, E.J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., Chen, W.: Lora: Low-rank adaptation of large language models. arXiv preprint arXiv:2106.09685 (2021). URL <https://arxiv.org/abs/2106.09685>
15. Kaplan, J., McCandlish, S., Henighan, T., Brown, T.B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., Amodei, D.: Scaling laws for neural language models. arXiv preprint arXiv:2001.08361 (2020). URL <https://arxiv.org/abs/2001.08361>
16. Karpukhin, V., Oguz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., Yih, W.t.: Dense passage retrieval for open-domain question answering. arXiv preprint arXiv:2004.04906 (2020). URL <https://arxiv.org/abs/2004.04906>
17. Kimi Team, Bai, Y., Bao, Y., Charles, Y., Chen, C., Chen, G., Chen, H., Chen, J., Chen, N., Chen, R., et al.: Kimi k2: Open agentic intelligence. arXiv preprint arXiv:2507.20534 (2025). URL <https://arxiv.org/abs/2507.20534>

18. Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C.H., Gonzalez, J.E., Zhang, H., Stoica, I.: Efficient memory management for large language model serving with pagedattention. arXiv preprint arXiv:2309.06180 (2023). URL <https://arxiv.org/abs/2309.06180>
19. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Kuttler, H., Lewis, M., Yih, W.t., Rocktaschel, T., et al.: Retrieval-augmented generation for knowledge-intensive nlp tasks. arXiv preprint arXiv:2005.11401 (2020). URL <https://arxiv.org/abs/2005.11401>
20. Lyu, Q., Havaldar, S., Stein, A., Zhang, L., Rao, D., Wong, E., Apidianaki, M., Callison-Burch, C.: Faithful chain-of-thought reasoning. arXiv preprint arXiv:2301.13379 (2023). URL <https://arxiv.org/abs/2301.13379>
21. Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C.L., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., et al.: Training language models to follow instructions with human feedback. arXiv preprint arXiv:2203.02155 (2022). URL <https://arxiv.org/abs/2203.02155>
22. Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., Sutskever, I.: Language models are unsupervised multitask learners. OpenAI technical report (2019). URL https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf
23. Rafailov, R., Sharma, A., Mitchell, E., Ermon, S., Manning, C.D., Finn, C.: Direct preference optimization: Your language model is secretly a reward model. arXiv preprint arXiv:2305.18290 (2023). URL <https://arxiv.org/abs/2305.18290>
24. Shazeer, N.: Fast transformer decoding: One write-head is all you need. arXiv preprint arXiv:1911.02150 (2019). URL <https://arxiv.org/abs/1911.02150>
25. Shi, W., Ajith, A., Xia, M., Huang, Y., Liu, D., Blevins, T., Chen, D., Zettlemoyer, L.: Detecting pretraining data from large language models. arXiv preprint arXiv:2310.16789 (2023). URL <https://arxiv.org/abs/2310.16789>
26. Su, J., Lu, Y., Pan, S., Murtadha, A., Wen, B., Liu, Y.: Roformer: Enhanced transformer with rotary position embedding. arXiv preprint arXiv:2104.09864 (2021). URL <https://arxiv.org/abs/2104.09864>
27. Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.A., Lacroix, T., Roziere, B., Goyal, N., Hambro, E., Azhar, F., et al.: Llama: Open and efficient foundation language models. arXiv preprint arXiv:2302.13971 (2023). URL <https://arxiv.org/abs/2302.13971>
28. Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N., Batra, S., Bhargava, P., Bhosale, S., et al.: Llama 2: Open foundation and fine-tuned chat models. arXiv preprint arXiv:2307.09288 (2023). URL <https://arxiv.org/abs/2307.09288>
29. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., Polosukhin, I.: Attention is all you need. arXiv preprint arXiv:1706.03762 (2017). URL <https://arxiv.org/abs/1706.03762>
30. Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E.H., Le, Q.V., Zhou, D.: Chain-of-thought prompting elicits reasoning in large language models. arXiv preprint arXiv:2201.11903 (2022). URL <https://arxiv.org/abs/2201.11903>
31. Wen, X., Liu, Z., Zheng, S., Ye, S., Wu, Z., Wang, Y., Xu, Z., Liang, X., Li, J., Miao, Z., et al.: Reinforcement learning with verifiable rewards implicitly incentivizes correct reasoning in base llms. arXiv preprint arXiv:2506.14245 (2025). URL <https://arxiv.org/abs/2506.14245>
32. Xu, C., Guan, S., Greene, D., Kechadi, M.T.: Benchmark data contamination of large language models: A survey. arXiv preprint arXiv:2406.04244 (2024). URL <https://arxiv.org/abs/2406.04244>
33. Yang, Z., Guo, X., Zhang, T., Xu, H., Li, B.: Test-time scaling of llms: A survey from a subproblem structure perspective. arXiv preprint arXiv:2511.14772 (2025). URL <https://arxiv.org/abs/2511.14772>
34. Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T.L., Cao, Y., Narasimhan, K.: Tree of thoughts: Deliberate problem solving with large language models. arXiv preprint arXiv:2305.10601 (2023). URL <https://arxiv.org/abs/2305.10601>
35. Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., Cao, Y.: React: Synergizing reasoning and acting in language models. arXiv preprint arXiv:2210.03629 (2023). URL <https://arxiv.org/abs/2210.03629>